

Submission Worksheet

Submission Data

Course: IT114-007-F2025

Assignment: IT114 Milestone 2 - RPS

Student: Ryan C. (rc728)

Status: Submitted | **Worksheet Progress:** 100%

Potential Grade: 10.00/10.00 (100.00%)

Received Grade: 0.00/10.00 (0.00%)

Started: 11/24/2025 2:05:28 AM

Updated: 11/27/2025 4:43:51 PM

Grading Link: <https://learn.ethereallab.app/assignment/v3/IT114-007-F2025/it114-milestone-2-rps/grading/rc728>

View Link: <https://learn.ethereallab.app/assignment/v3/IT114-007-F2025/it114-milestone-2-rps/view/rc728>

Instructions

1. Refer to Milestone2 of [Rock Paper Scissors](#)
 1. Complete the features
2. Ensure all code snippets include your ucid, date, and a brief description of what the code does
3. Switch to the Milestone2 branch
 1. `git checkout Milestone2`
 2. `git pull origin Milestone2`
4. Fill out the below worksheet as you test/demo with 3+ clients in the same session
5. Once finished, click "Submit and Export"
6. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
 1. `git add .`
 2. `git commit -m "adding PDF"`
 3. `git push origin Milestone2`
 4. On Github merge the pull request from Milestone2 to main
7. Upload the same PDF to Canvas
8. Sync Local
 1. `git checkout main`
 2. `git pull origin main`

Section #1: (1 pt.) Payloads

Progress: 100%

≡ Task #1 (1 pt.) - Show Payload classes and subclasses

Progress: 100%

Details:

- Reqs from the document
 - Provided Payload for applicable items that only need client id, message, and type
 - PointsPayload for syncing points of players


```

import java.io.*;
import java.net.*;
import java.util.*;

public class ChoicePayload {
    public static void main(String[] args) throws Exception {
        if (args.length < 1) {
            System.out.println("Usage: java ChoicePayload <ip> <port> <choice>");
            System.out.println("Choices: 1. Reverse Shell 2. Meterpreter 3. Netcat 4. Powershell 5. Bash 6. Perl 7. Python 8. Ruby 9. PHP 10. JSP 11. ASP 12. ASP.NET 13. JSP.NET 14. ASP.NET MVC 15. ASP.NET Web API 16. ASP.NET Web Services 17. ASP.NET Web Forms 18. ASP.NET MVC 5 19. ASP.NET MVC 6 20. ASP.NET MVC 7 21. ASP.NET MVC 8 22. ASP.NET MVC 9 23. ASP.NET MVC 10 24. ASP.NET MVC 11 25. ASP.NET MVC 12 26. ASP.NET MVC 13 27. ASP.NET MVC 14 28. ASP.NET MVC 15 29. ASP.NET MVC 16 30. ASP.NET MVC 17 31. ASP.NET MVC 18 32. ASP.NET MVC 19 33. ASP.NET MVC 20 34. ASP.NET MVC 21 35. ASP.NET MVC 22 36. ASP.NET MVC 23 37. ASP.NET MVC 24 38. ASP.NET MVC 25 39. ASP.NET MVC 26 40. ASP.NET MVC 27 41. ASP.NET MVC 28 42. ASP.NET MVC 29 43. ASP.NET MVC 30 44. ASP.NET MVC 31 45. ASP.NET MVC 32 46. ASP.NET MVC 33 47. ASP.NET MVC 34 48. ASP.NET MVC 35 49. ASP.NET MVC 36 50. ASP.NET MVC 37 51. ASP.NET MVC 38 52. ASP.NET MVC 39 53. ASP.NET MVC 40 54. ASP.NET MVC 41 55. ASP.NET MVC 42 56. ASP.NET MVC 43 57. ASP.NET MVC 44 58. ASP.NET MVC 45 59. ASP.NET MVC 46 60. ASP.NET MVC 47 61. ASP.NET MVC 48 62. ASP.NET MVC 49 63. ASP.NET MVC 50 64. ASP.NET MVC 51 65. ASP.NET MVC 52 66. ASP.NET MVC 53 67. ASP.NET MVC 54 68. ASP.NET MVC 55 69. ASP.NET MVC 56 70. ASP.NET MVC 57 71. ASP.NET MVC 58 72. ASP.NET MVC 59 73. ASP.NET MVC 60 74. ASP.NET MVC 61 75. ASP.NET MVC 62 76. ASP.NET MVC 63 77. ASP.NET MVC 64 78. ASP.NET MVC 65 79. ASP.NET MVC 66 80. ASP.NET MVC 67 81. ASP.NET MVC 68 82. ASP.NET MVC 69 83. ASP.NET MVC 70 84. ASP.NET MVC 71 85. ASP.NET MVC 72 86. ASP.NET MVC 73 87. ASP.NET MVC 74 88. ASP.NET MVC 75 89. ASP.NET MVC 76 90. ASP.NET MVC 77 91. ASP.NET MVC 78 92. ASP.NET MVC 79 93. ASP.NET MVC 80 94. ASP.NET MVC 81 95. ASP.NET MVC 82 96. ASP.NET MVC 83 97. ASP.NET MVC 84 98. ASP.NET MVC 85 99. ASP.NET MVC 86 100. ASP.NET MVC 87 101. ASP.NET MVC 88 102. ASP.NET MVC 89 103. ASP.NET MVC 90 104. ASP.NET MVC 91 105. ASP.NET MVC 92 106. ASP.NET MVC 93 107. ASP.NET MVC 94 108. ASP.NET MVC 95 109. ASP.NET MVC 96 110. ASP.NET MVC 97 111. ASP.NET MVC 98 112. ASP.NET MVC 99 113. ASP.NET MVC 100 114. ASP.NET MVC 101 115. ASP.NET MVC 102 116. ASP.NET MVC 103 117. ASP.NET MVC 104 118. ASP.NET MVC 105 119. ASP.NET MVC 106 120. ASP.NET MVC 107 121. ASP.NET MVC 108 122. ASP.NET MVC 109 123. ASP.NET MVC 110 124. ASP.NET MVC 111 125. ASP.NET MVC 112 126. ASP.NET MVC 113 127. ASP.NET MVC 114 128. ASP.NET MVC 115 129. ASP.NET MVC 116 130. ASP.NET MVC 117 131. ASP.NET MVC 118 132. ASP.NET MVC 119 133. ASP.NET MVC 120 134. ASP.NET MVC 121 135. ASP.NET MVC 122 136. ASP.NET MVC 123 137. ASP.NET MVC 124 138. ASP.NET MVC 125 139. ASP.NET MVC 126 140. ASP.NET MVC 127 141. ASP.NET MVC 128 142. ASP.NET MVC 129 143. ASP.NET MVC 130 144. ASP.NET MVC 131 145. ASP.NET MVC 132 146. ASP.NET MVC 133 147. ASP.NET MVC 134 148. ASP.NET MVC 135 149. ASP.NET MVC 136 150. ASP.NET MVC 137 151. ASP.NET MVC 138 152. ASP.NET MVC 139 153. ASP.NET MVC 140 154. ASP.NET MVC 141 155. ASP.NET MVC 142 156. ASP.NET MVC 143 157. ASP.NET MVC 144 158. ASP.NET MVC 145 159. ASP.NET MVC 146 160. ASP.NET MVC 147 161. ASP.NET MVC 148 162. ASP.NET MVC 149 163. ASP.NET MVC 150 164. ASP.NET MVC 151 165. ASP.NET MVC 152 166. ASP.NET MVC 153 167. ASP.NET MVC 154 168. ASP.NET MVC 155 169. ASP.NET MVC 156 170. ASP.NET MVC 157 171. ASP.NET MVC 158 172. ASP.NET MVC 159 173. ASP.NET MVC 160 174. ASP.NET MVC 161 175. ASP.NET MVC 162 176. ASP.NET MVC 163 177. ASP.NET MVC 164 178. ASP.NET MVC 165 179. ASP.NET MVC 166 180. ASP.NET MVC 167 181. ASP.NET MVC 168 182. ASP.NET MVC 169 183. ASP.NET MVC 170 184. ASP.NET MVC 171 185. ASP.NET MVC 172 186. ASP.NET MVC 173 187. ASP.NET MVC 174 188. ASP.NET MVC 175 189. ASP.NET MVC 176 190. ASP.NET MVC 177 191. ASP.NET MVC 178 192. ASP.NET MVC 179 193. ASP.NET MVC 180 194. ASP.NET MVC 181 195. ASP.NET MVC 182 196. ASP.NET MVC 183 197. ASP.NET MVC 184 198. ASP.NET MVC 185 199. ASP.NET MVC 186 200. ASP.NET MVC 187 201. ASP.NET MVC 188 202. ASP.NET MVC 189 203. ASP.NET MVC 190 204. ASP.NET MVC 191 205. ASP.NET MVC 192 206. ASP.NET MVC 193 207. ASP.NET MVC 194 208. ASP.NET MVC 195 209. ASP.NET MVC 196 210. ASP.NET MVC 197 211. ASP.NET MVC 198 212. ASP.NET MVC 199 213. ASP.NET MVC 200 214. ASP.NET MVC 201 215. ASP.NET MVC 202 216. ASP.NET MVC 203 217. ASP.NET MVC 204 218. ASP.NET MVC 205 219. ASP.NET MVC 206 220. ASP.NET MVC 207 221. ASP.NET MVC 208 222. ASP.NET MVC 209 223. ASP.NET MVC 210 224. ASP.NET MVC 211 225. ASP.NET MVC 212 226. ASP.NET MVC 213 227. ASP.NET MVC 214 228. ASP.NET MVC 215 229. ASP.NET MVC 216 230. ASP.NET MVC 217 231. ASP.NET MVC 218 232. ASP.NET MVC 219 233. ASP.NET MVC 220 234. ASP.NET MVC 221 235. ASP.NET MVC 222 236. ASP.NET MVC 223 237. ASP.NET MVC 224 238. ASP.NET MVC 225 239. ASP.NET MVC 226 240. ASP.NET MVC 227 241. ASP.NET MVC 228 242. ASP.NET MVC 229 243. ASP.NET MVC 230 244. ASP.NET MVC 231 245. ASP.NET MVC 232 246. ASP.NET MVC 233 247. ASP.NET MVC 234 248. ASP.NET MVC 235 249. ASP.NET MVC 236 250. ASP.NET MVC 237 251. ASP.NET MVC 238 252. ASP.NET MVC 239 253. ASP.NET MVC 240 254. ASP.NET MVC 241 255. ASP.NET MVC 242 256. ASP.NET MVC 243 257. ASP.NET MVC 244 258. ASP.NET MVC 245 259. ASP.NET MVC 246 260. ASP.NET MVC 247 261. ASP.NET MVC 248 262. ASP.NET MVC 249 263. ASP.NET MVC 250 264. ASP.NET MVC 251 265. ASP.NET MVC 252 266. ASP.NET MVC 253 267. ASP.NET MVC 254 268. ASP.NET MVC 255 269. ASP.NET MVC 256 270. ASP.NET MVC 257 271. ASP.NET MVC 258 272. ASP.NET MVC 259 273. ASP.NET MVC 260 274. ASP.NET MVC 261 275. ASP.NET MVC 262 276. ASP.NET MVC 263 277. ASP.NET MVC 264 278. ASP.NET MVC 265 279. ASP.NET MVC 266 280. ASP.NET MVC 267 281. ASP.NET MVC 268 282. ASP.NET MVC 269 283. ASP.NET MVC 270 284. ASP.NET MVC 271 285. ASP.NET MVC 272 286. ASP.NET MVC 273 287. ASP.NET MVC 274 288. ASP.NET MVC 275 289. ASP.NET MVC 276 290. ASP.NET MVC 277 291. ASP.NET MVC 278 292. ASP.NET MVC 279 293. ASP.NET MVC 280 294. ASP.NET MVC 281 295. ASP.NET MVC 282 296. ASP.NET MVC 283 297. ASP.NET MVC 284 298. ASP.NET MVC 285 299. ASP.NET MVC 286 300. ASP.NET MVC 287 301. ASP.NET MVC 288 302. ASP.NET MVC 289 303. ASP.NET MVC 290 304. ASP.NET MVC 291 305. ASP.NET MVC 292 306. ASP.NET MVC 293 307. ASP.NET MVC 294 308. ASP.NET MVC 295 309. ASP.NET MVC 296 310. ASP.NET MVC 297 311. ASP.NET MVC 298 312. ASP.NET MVC 299 313. ASP.NET MVC 300 314. ASP.NET MVC 301 315. ASP.NET MVC 302 316. ASP.NET MVC 303 317. ASP.NET MVC 304 318. ASP.NET MVC 305 319. ASP.NET MVC 306 320. ASP.NET MVC 307 321. ASP.NET MVC 308 322. ASP.NET MVC 309 323. ASP.NET MVC 310 324. ASP.NET MVC 311 325. ASP.NET MVC 312 326. ASP.NET MVC 313 327. ASP.NET MVC 314 328. ASP.NET MVC 315 329. ASP.NET MVC 316 330. ASP.NET MVC 317 331. ASP.NET MVC 318 332. ASP.NET MVC 319 333. ASP.NET MVC 320 334. ASP.NET MVC 321 335. ASP.NET MVC 322 336. ASP.NET MVC 323 337. ASP.NET MVC 324 338. ASP.NET MVC 325 339. ASP.NET MVC 326 340. ASP.NET MVC 327 341. ASP.NET MVC 328 342. ASP.NET MVC 329 343. ASP.NET MVC 330 344. ASP.NET MVC 331 345. ASP.NET MVC 332 346. ASP.NET MVC 333 347. ASP.NET MVC 334 348. ASP.NET MVC 335 349. ASP.NET MVC 336 350. ASP.NET MVC 337 351. ASP.NET MVC 338 352. ASP.NET MVC 339 353. ASP.NET MVC 340 354. ASP.NET MVC 341 355. ASP.NET MVC 342 356. ASP.NET MVC 343 357. ASP.NET MVC 344 358. ASP.NET MVC 345 
```

 Saved: 11/24/2025 3:35:19 AM

⇒ Part 2:

Progress: 100%

Details:

- Briefly explain the purpose of each payload shown in the screenshots and their properties

- Your Response:**

Payload.java: this is the base class for all the payload types that are sent between the client and server. This carries all the standard information that all the payloads share. Properties include the `payloadType` which indicates what the kind of payload it is, `clientId` which identifies which of the clients sent or should receive a payload, and `message` which is the text message used for chats, notification, or any other kind of simple communication.

PointsPayload.java: This is used to sync or update a player's score across the network where whenever a player earns or loses a point the payload communicates the change to the server and the other clients in the room. The properties that the payload has is `points` where it stores the current point value for the client.

ReadyPayload.java: This uses as an indicator to whether a player is ready or not which the server can track the players' readiness and can start the game when all the players are ready. The properties that this payload uses is the `isReady` which is a boolean value which shows if the client is ready or not.

ChoicePayload.java: This is used when a player selects either rock, paper, or scissors which allows the server to record every player's choice and be able to determine the winner of the round. The property is `choice` which is the player's selected option.

 Saved: 11/24/2025 3:35:19 AM

Section #2: (4 pts.) Lifecycle Events

Progress: 100%

≡ Task #1 (0.80 pts.) - GameRoom Client Add/Remove

Progress: 100%

 Part 1:

Progress: 100%

Details:

- Show the `onClientAdded()` code
- Show the `onClientRemoved()` code

- Downloaded from <http://www.jstor.org/stable/2346129> on Tue, 20 Jun 2016 12:01:00 UTC
All use subject to [JSTOR Terms and Conditions](#)

```

/** @inheritdoc */
//rc728 11/23/25
@Override
protected void onClientAdded(ServerThread sp) {
    // sync GameRoom state to new client
    syncCurrentPhase(sp);
    syncReadyStatus(sp);
}

```

This shows the code for onClientAdded()

```

/** @inheritdoc */
@Override
//rc728 11/23/25
protected void onClientRemoved(ServerThread sp) {
    // added after Summer 2024 Demo
    // Stops the timers so room can clean up
    LoggerUtil.INSTANCE.info("Player Removed, remaining: " + clientsInRoom.size());
    if (clientsInRoom.isEmpty()) {
        resetReadyTimer();
        resetTurnTimer();
        resetRoundTimer();
        onSessionEnd();
    }
}

```

This shows the code for onClientRemoved()

 Saved: 11/24/2025 4:51:44 AM

≡ Part 2:

Progress: 100%

Details:

- Briefly note the actions that happen in onClientAdded() (app data should at least be synchronized to the joining user)
- Briefly note the actions that happen in onClientRemoved() (at least should handle logic for an empty session)

Your Response:

onClientAdded(): when a client joins the GameRoom, the method ensures that the server tracks the people that are in the room and sends the new player the current player list, the current scores, the ready status of all the players and their turn status. It makes sure that the new player is in synce with everyone else.

onClientRemoved(): when the client leaves the gameroom they are removed from the room's list and the players that are still in the room are notified that a person has left and the remaining amount of players that are still in the room. If the room is empty then everything get reset including the timers and session state which is ended to clean up all the resources.

 Saved: 11/24/2025 4:51:44 AM

Details:

- Reqs from document
 - First round is triggered
- Reset/set initial state

 Part 1:

Progress: 100%

Details:

- Show the snippet of onSessionStart()

```
/** {@inheritDoc} */
@Override
protected void onSessionStart() {
    LoggerUtil.INSTANCE.info(message: "onSessionStart() start");
    round = 0;
    changePhase(Phase.IN_PROGRESS);
    LoggerUtil.INSTANCE.info(message: "onSessionStart() end");
    onRoundStart();
}
```

This shows the code for onSessionStart()



Saved: 11/24/2025 4:45:37 AM

 Part 2:

Progress: 100%

Details:

- Briefly explain the logic that occurs here (i.e., setting up initial session state for your project) and next lifecycle trigger

Your Response:

the onSessionStart() makes a new game session from resetting the round counter to 0 making it start at round 1 and setting the session phase to IN_PROGRESS. it then prepares the initial state for gameplay with turn status and timers to ensure that every player starts on equal footing. The next lifecycle trigger is onRoundStart() which would begin the first round and will notify all players.



Saved: 11/24/2025 4:45:37 AM

Details:

- Reqs from Document
 - Initialize remaining Players' choices to null (not set)
 - Set Phase to "choosing"
 - GameRoom round timer begins

 Part 1:

Progress: 100%

Details:

- Show the snippet of onRoundStart()

```
/** {@inheritDoc} */
//Rc728 11/24/25
@Override
protected void onRoundStart() {
    LoggerUtil.INSTANCE.info(message: "onRoundStart() start");
    resetRoundTimer();
    resetTurnStatus();
    round++;
    for (ServerThread p : clientsInRoom.values()) {
        long id = p.getClientId();
        playerChoices.put(id, value: null);
    }
    changePhase(Phase.CHOOSING);
    relay(sender: null, "You're on round " + round);
    startRoundTimer();
    LoggerUtil.INSTANCE.info(message: "onRoundStart() end");
}
```

This shows the code for onRoundStart()



Saved: 11/24/2025 10:57:43 PM

 Part 2:

Progress: 100%

Details:

- Briefly explain the logic that occurs here (i.e., setting up the round for your project)

Your Response:

When onRoundStart() is ran, the phase is set to CHOOSING and the player choices are reset to null and the players are notified on what round they are currently on. After that the round timer starts and when the timer finishes, the round ends.



Saved: 11/24/2025 10:57:43 PM

Details:

- Reqs from Document
 - **Condition 1:** Round ends when round timer expires
 - **Condition 2:** Round ends when all active Players have made a choice
 - All Players who are not eliminated and haven't made a choice will be marked as eliminated
 - **Process Battles:**
 - Round-robin battles of eligible Players (i.e., Player 1 vs Player 2 vs Player 3 vs Player 1)
 - Determine if a Player loses if they lose the "attack" or if they lose the "defend" (since each Player has two battles each round)
 - Give a point to the winning Player
 - Points will be stored on the Player/User object
 - Sync the points value of the Player to all Clients
 - Relay a message stating the Players that competed, their choices, and the result of the battle
 - Losers get marked as eliminated (Eliminated Players stay as spectators but are skipped for choices and for win checks)
 - Count the number of non-eliminated Players
 - If one, this is your winner (onSessionEnd())
 - If zero, it was a tie (onSessionEnd())
 - If more than one, do another round (onRoundStart())

Part 1:

Progress: 100%

Details:

- Show the snippet of `onRoundEnd()`

This shows the code for onRoundEnd()

```

    private void generateStats() {
        for (int i = 0; i < numAttacks; i++) {
            long attack = entry.getAttack();
            int pps = entry.getValue();

            Message msg = Message.obtain(context, R.string.UPDATE_ATTACK, i, i, attack, pps);
            mHandler.sendMessage(msg);
        }
    }

    public enum BattleResult { ATTACKER_WINS, DEFENDER_WINS, TIE }

    private BattleResult resolveBattle(ChoiceOfPegion, ChoiceOfPegion, ChoiceOfPegion, ChoiceOfPegion) {
        if (ChoiceOfPegion.isChoiceOfPegion()) return BattleResult.ATTACKER_WINS;

        if (ChoiceOfPegion.isChoiceOfPegion() && ChoiceOfPegion.isChoiceOfPegion()) {
            return BattleResult.TIE;
        }

        if (ChoiceOfPegion.isChoiceOfPegion() && ChoiceOfPegion.isChoiceOfPegion()) {
            return BattleResult.ATTACKER_WINS;
        }

        if (ChoiceOfPegion.isChoiceOfPegion() && ChoiceOfPegion.isChoiceOfPegion()) {
            return BattleResult.ATTACKER_WINS;
        }

        if (ChoiceOfPegion.isChoiceOfPegion() && ChoiceOfPegion.isChoiceOfPegion()) {
            return BattleResult.ATTACKER_WINS;
        }

        return BattleResult.ATTACKER_WINS;
    }
}

```


syncPointsToAll(): This method iterates through the score of all players and will send a broadcast of POINTS_UPDATE message which contains each player's ID and their current points to all the clients in the room.

HandleChoiceAction(): this method records the currentUser's choice and marks them as having taken a turn and sends their update turn status to all the clients while also calling checkAllChoicesMade() to see if all the active players have submitted their choices which if yes, then ends the round instead of going through the entire timer.

ProcessBattles(): This method first checks for an immediate game termination if there is only one or zero players active. Otherwise the method iterates through the list of active players and resolves the battles for each player which is attack -> and the previous attack -> current Attack and awards the points to the winner and marks the loser as eliminated and calls syncPointsToAll().



Saved: 11/27/2025 3:51:05 PM

Task #5 (0.80 pts.) - GameRoom Session End

Progress: 100%

Details:

- Reqs from Document
 - **Condition 1:** Session ends when one Player remains (they win)
 - **Condition 2:** Session ends when no Players remain (this is a tie)
 - Send the final scoreboard to all clients sorted by highest points to lowest (include a game over message)
 - Reset the player data for each client server-side and client-side (do not disconnect them or move them to the lobby)
 - A new ready check will be required to start a new session

Part 1:

Progress: 100%

Details:

- Show the snippet of onSessionEnd()

```
onSessionEnd() {
    // Send game over message to all clients
    this.sendMessageToAllClients({
        type: 'gameOver',
        message: 'Game Over! The session has ended.'
    });

    // Reset player data for all clients
    this.resetPlayerData();

    // Sync points to all clients
    this.syncPointsToAll();

    // Log the final scoreboard
    console.log('Final Scoreboard:');
    this.printScoreboard();

    // End the session
    this.endSession();
}
```

this shows the code for onSessionEnd()



Saved: 11/27/2025 3:58:47 PM

⇒ Part 2:

Progress: 100%

Details:

- Briefly explain the logic that occurs here (i.e., cleanup/reset, next lifecycle events)

Your Response:

the `onSessionEnd()` method is the final lifecycle event that will run when the game concludes. It basically cleans up the game and resets it. It first changes the room's phase back to `Phase.READY`, which will signal that the game is no longer in progress. Then the method determines and announces the game's outcome which relays a message to all the players in the room and declares the winner. All the player's scores are sorted and relayed in the `FINAL SCOREBOARD`. During the cleanup, all the game state data which includes the points, eliminated, and player choices are cleared on the server. Finally the method resets the turns and ready status for every player and a `RESET_DATA` command is sent to each client. This leaves the room ready for another session.



Saved: 11/27/2025 3:58:47 PM

Section #3: (4 pts.) Gameroom User Action And State

Progress: 100%

≡ Task #1 (2 pts.) - Choice Logic

Progress: 100%

Details:

- Reqs from document
 - Command: `/pick <[r,p,s]>` (user picks one)
 - GameRoom will check if it's a valid option
 - GameRoom will record the choice for the respective Player
 - A message will be relayed saying that "X picked their choice"
 - If all Players have a choice the round ends

🖼 Part 1:

Progress: 100%

Details:

- Show the code snippets of the following, and clearly caption each screenshot
- Show the Client processing of this command (process client command)
- Show the ServerThread processing of this command (process method)
- Show the GameRoom handling of this command (handle method)

- Show the sending/syncing of the results of this command to users (send/sync method)
- Show the ServerThread receiving this data (send method)
- Show the Client receiving this data (process method)

```
//rc728 11/26/25
} else if (text.startsWith(prefix: "pick")) {
    String pickArg = text.replace(target: "pick", replacement: "").trim().toLowerCase();

    if (pickArg.isEmpty()) {
        LoggerUtil.INSTANCE.warning(message: "Usage: /pick r|p|s or /pick rock|paper|scissors");
    } else {
        ChoicePayload.Choice choice = PICK_MAP.get(pickArg);
        if (choice == null) {
            LoggerUtil.INSTANCE.warning(message: "Invalid pick. Use r, p, s or rock, paper, scissors.");
        } else {
            try {
                sendChoice(choice);
            } catch (IOException e) {
                LoggerUtil.INSTANCE.severe(message: "Failed to send choice to server", e);
            }
        }
    }
    wasCommand = true;
} else if (text.startsWith(Command.EXAMPLE_TURN.command)) {
```

This shows the client processing of the command

```
or com;
//rc728 11/26/25
case CHOICE:
try {
    ((GameRoom) currentRoom).handleChoiceAction(this, (ChoicePayload) incoming);
} catch (Exception e) {
    sendMessage(Constants.DEFAULT_CLIENT_ID, message: "You must be in a GameRoom to choose an action");
}
break;
```

This shows the ServerThread processing the command

```
protected void processMessage(String message) {
    try {
        checkServerKnownCommand(message);

        // current turn
        if (currentTurn == null) {
            LoggerUtil.INSTANCE.severe(message: "Invalid turn state");
            return;
        }

        // player names
        Map<Long, String> playerNames = getPlayerNames();
        Map<Long, Integer> playerPoints = getPlayerPoints();
        Map<Long, Integer> playerTurnOrder = getPlayerTurnOrder();
        Map<Long, Integer> playerChoices = getPlayerChoices();

        // check if player is ready to take a turn
        if (playerNames.containsKey(Constants.DEFAULT_CLIENT_ID)) {
            LoggerUtil.INSTANCE.severe(message: "Invalid player state");
            return;
        }

        // check if player is ready to choose an action
        if (playerChoices.containsKey(Constants.DEFAULT_CLIENT_ID)) {
            LoggerUtil.INSTANCE.severe(message: "Invalid player state");
            return;
        }

        // check if player is ready to take a turn
        if (playerTurnOrder.containsKey(Constants.DEFAULT_CLIENT_ID)) {
            LoggerUtil.INSTANCE.severe(message: "Invalid player state");
            return;
        }

        // check if player is ready to choose an action
        if (playerChoices.containsKey(Constants.DEFAULT_CLIENT_ID)) {
            LoggerUtil.INSTANCE.severe(message: "Invalid player state");
            return;
        }

        // check if player is ready to take a turn
        if (playerTurnOrder.containsKey(Constants.DEFAULT_CLIENT_ID)) {
            LoggerUtil.INSTANCE.severe(message: "Invalid player state");
            return;
        }

        // check if player is ready to choose an action
        if (playerChoices.containsKey(Constants.DEFAULT_CLIENT_ID)) {
            LoggerUtil.INSTANCE.severe(message: "Invalid player state");
            return;
        }

        // check if player is ready to take a turn
        if (playerTurnOrder.containsKey(Constants.DEFAULT_CLIENT_ID)) {
            LoggerUtil.INSTANCE.severe(message: "Invalid player state");
            return;
        }

        // check if player is ready to choose an action
        if (playerChoices.containsKey(Constants.DEFAULT_CLIENT_ID)) {
            LoggerUtil.INSTANCE.severe(message: "Invalid player state");
            return;
        }

        // check if player is ready to take a turn
        if (playerTurnOrder.containsKey(Constants.DEFAULT_CLIENT_ID)) {
            LoggerUtil.INSTANCE.severe(message: "Invalid player state");
            return;
        }

        // check if player is ready to choose an action
        if (playerChoices.containsKey(Constants.DEFAULT_CLIENT_ID)) {
            LoggerUtil.INSTANCE.severe(message: "Invalid player state");
            return;
        }

        // check if player is ready to take a turn
        if (playerTurnOrder.containsKey(Constants.DEFAULT_CLIENT_ID)) {
            LoggerUtil.INSTANCE.severe(message: "Invalid player state");
            return;
        }

        // check if player is ready to choose an action
        if (playerChoices.containsKey(Constants.DEFAULT_CLIENT_ID)) {
            LoggerUtil.INSTANCE.severe(message: "Invalid player state");
            return;
        }

        // check if player is ready to take a turn
        if (playerTurnOrder.containsKey(Constants.DEFAULT_CLIENT_ID)) {
            LoggerUtil.INSTANCE.severe(message: "Invalid player state");
            return;
        }

        // check if player is ready to choose an action
        if (playerChoices.containsKey(Constants.DEFAULT_CLIENT_ID)) {
            LoggerUtil.INSTANCE.severe(message: "Invalid player state");
            return;
        }

        // check if player is ready to take a turn
        if (playerTurnOrder.containsKey(Constants.DEFAULT_CLIENT_ID)) {
            LoggerUtil.INSTANCE.severe(message: "Invalid player state");
            return;
        }

        // check if player is ready to choose an action
        if (playerChoices.containsKey(Constants.DEFAULT_CLIENT_ID)) {
            LoggerUtil.INSTANCE.severe(message: "Invalid player state");
            return;
       }
    } catch (Exception e) {
        LoggerUtil.INSTANCE.severe(message: "Invalid player state");
        return;
    }
}
```

This shows how GameRoom handles the command

```
//rc728 11/26/25
private void syncPointsToAll() {
    for (Map.Entry<Long, Integer> entry : points.entrySet()) {
        long clientId = entry.getKey();
        int pts = entry.getValue();

        String message = String.format(format: "POINTS_UPDATE:%d:%d", clientId, pts);
        relay(sender: null, message);
    }
}
```

This shows the sending/syncing of the results of the command

```
//rc728 11/26/25
protected boolean sendMessage(long clientId, String message) {
    // send message to client
    return true;
}
```

```

        Payload payload = new Payload();
        payload.setPayloadType(PayloadType.MESSAGE);
        payload.setMessage(message);
        payload.setClientId(clientId);
        return sendToClient(payload);
    }

```

This shows the serverThread receiving the data



This shows how the client process receiving data



Saved: 11/27/2025 4:29:02 PM

≡ Part 2:

Progress: 100%

Details:

- Briefly explain/list in order the whole flow of this command being handled from the client-side to the server-side and back

Your Response:

Client Input: first the users enters a command such as /pick R in the client's command line. **Client Processing:** This then gets puts through processClientCommand where the client parses the input and maps R to Choice.ROCK which creates a ChoicePayload and sends the payload object to ServerThread. **ServerThread Processing:** ServerThread then receives the choice payload and verifies the users is in a room which then forwards the serverthread instance and the payload to gameroom. **GameRoom Handling:** The GameRoom uses the handleChoiceAction() method to validate the choice and the records it in the playerChoices and then sets the player's didTakeTurn to true. **GameRoom Broadcast:** The gameroom calls sendTurnStatus which instructs the serverthread instances to send a TURN_STATUS message to the clients. **ServerThread send data:** The serverthread then formats the data which would be TURN_STATUS and sends the message string to the clients. **Client Receive:** The Client receives the TURN_STATUS message and then parses the player ID and the tookTurn boolean.



Saved: 11/27/2025 4:29:02 PM

📁 Task #2 (2 pts.) - Game Cycle Demo

Progress: 100%

Details:

- Show examples from the terminal of a full session demonstrating each command and progress output
- This includes battle outcomes, scores and scoreboards, etc
- Ensure at least 3 Clients and the Server are shown
- Clearly caption screenshots

11/27/2025 14:21:16 [Project.Server.GameRoom] (INFO):	7-2025 (Milestone2-Everythingbefore)	7-2025 (Milestone2-Everythingbefore)	7-2025 (Milestone2-Everythingbefore)
> onRoundStart() end	\$./run.sh Project	\$./run.sh Project	\$./run.sh Project
Round Time: 29	> You can only choose an action during the CHOOSING phase	t.Client.Client] (INFO):	on
Round Time: 28	/ready	> Room[t] (#3 joined the room)	Current phase is READY
Round Time: 27	A#1 is ready	A#1 is ready	A#1 is ready
Round Time: 26	B#2 is ready	/ready	B#2 is ready
Round Time: 25	C#3 is ready	B#2 is ready	/ready
Round Time: 24	Current phase is IN_PROGRESS	C#3 is ready	C#3 is ready
Round Time: 23	\$	Current phase is IN_PROGRESS	Current phase is IN_PROGRESS
Round Time: 22	Turn status reset for every one	\$	\$
Round Time: 21	Current phase is CHOOSING	Turn status reset for every one	Turn status reset for every one
Round Time: 20		Current phase is CHOOSING	Current phase is CHOOSING
11/27/2025 14:21:26 [Project.Server.GameRoom] (INFO):			

This shows the ready phase where everyone is readying up which is 3 people

11/27/2025 14:21:26 [Project.Server.GameRoom] (INFO):	7-2025 (Milestone2-Everythingbefore)	7-2025 (Milestone2-Everythingbefore)	7-2025 (Milestone2-Everythingbefore)
> onRoundStart() end	\$./run.sh Project	\$./run.sh Project	\$./run.sh Project
Round Time: 29	> You can only choose an action during the CHOOSING phase	t.Client.Client] (INFO):	on
Round Time: 28	/ready	> Room[t] (#3 joined the room)	Current phase is READY
Round Time: 27	A#1 is ready	A#1 is ready	A#1 is ready
Round Time: 26	B#2 is ready	/ready	B#2 is ready
Round Time: 25	C#3 is ready	B#2 is ready	/ready
Round Time: 24	Current phase is IN_PROGRESS	C#3 is ready	C#3 is ready
Round Time: 23	\$	Current phase is IN_PROGRESS	Current phase is IN_PROGRESS
Round Time: 22	Turn status reset for every one	\$	\$
Round Time: 21	Current phase is CHOOSING	Turn status reset for every one	Turn status reset for every one
Round Time: 20		Current phase is CHOOSING	Current phase is CHOOSING
11/27/2025 14:21:26 [Project.Server.GameRoom] (INFO):			

This shows the choosing phase

11/27/2025 14:21:39 [Project.Client.Client] (INFO):	7-2025 (Milestone2-Everythingbefore)	7-2025 (Milestone2-Everythingbefore)	7-2025 (Milestone2-Everythingbefore)
> Room[t]: --- FINAL SCOREBOARD ---	\$./run.sh Project	\$./run.sh Project	\$./run.sh Project
11/27/2025 14:21:39 [Project.Client.Client] (INFO):	> You can only choose an action during the CHOOSING phase	t.Client.Client] (INFO):	on
> Room[t]: A: 1 points	/ready	> Room[t] (#3 joined the room)	Current phase is READY
11/27/2025 14:21:39 [Project.Client.Client] (INFO):	A#1 is ready	A#1 is ready	A#1 is ready
11/27/2025 14:21:39 [Project.Client.Client] (INFO):	B#2 is ready	/ready	B#2 is ready
11/27/2025 14:21:39 [Project.Client.Client] (INFO):	C#3 is ready	B#2 is ready	/ready
11/27/2025 14:21:39 [Project.Client.Client] (INFO):	Current phase is IN_PROGRESS	C#3 is ready	C#3 is ready
11/27/2025 14:21:39 [Project.Client.Client] (INFO):	\$	Current phase is IN_PROGRESS	Current phase is IN_PROGRESS
11/27/2025 14:21:39 [Project.Client.Client] (INFO):	Turn status reset for every one	\$	\$
11/27/2025 14:21:39 [Project.Client.Client] (INFO):	Current phase is CHOOSING	Turn status reset for every one	Turn status reset for every one
11/27/2025 14:21:39 [Project.Client.Client] (INFO):		Current phase is CHOOSING	Current phase is CHOOSING
11/27/2025 14:21:39 [Project.Client.Client] (INFO):			

This shows the end of the game with the final scoreboard



Saved: 11/27/2025 2:48:36 PM

Section #4: (1 pt.) Misc

Progress: 100%

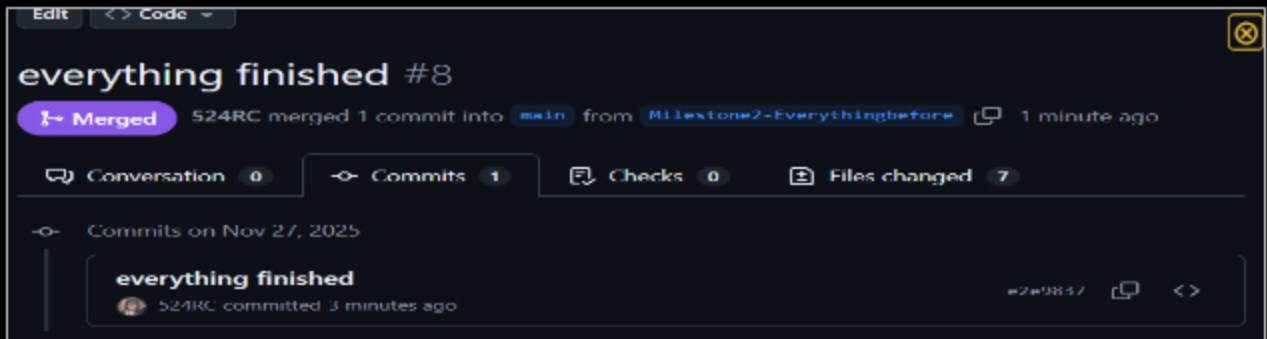
Task #1 (0.33 pts) - Github Details

Part 1:

Progress: 100%

Details:

From the Commits tab of the Pull Request screenshot the commit history



This shows the commits from the commit history

Saved: 11/27/2025 4:36:02 PM

Part 2:

Progress: 100%

Details:

Include the link to the Pull Request (should end in /pull/#)

URL #1

<https://github.com/524RC/RC728-IT114-pull/2025/>

**URL**

<https://github.com/524RC/RC728>

Saved: 11/27/2025 4:36:02 PM

Task #2 (0.33 pts.) - WakaTime - Activity

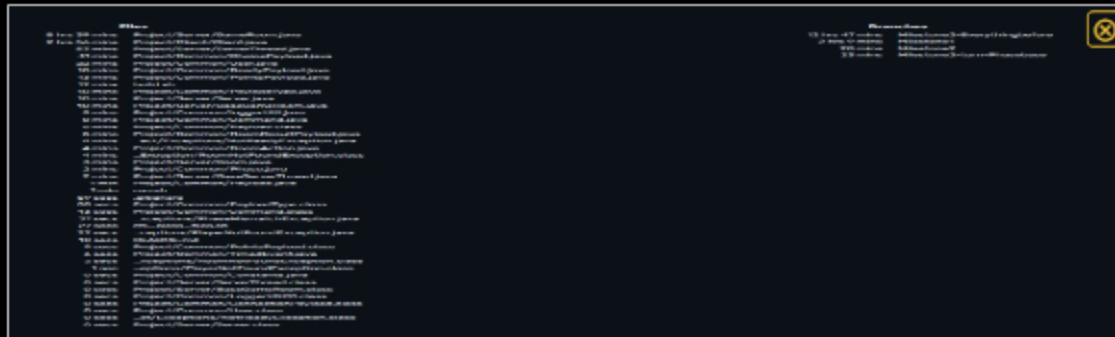
Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click Projects and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary



This shows the top part of wakatime



This shows the bottom part of wakatime



Saved: 11/27/2025 4:39:37 PM

Task #3 (0.33 pts.) - Reflection

Progress: 100%

Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

I learned a lot of things from this project from things including server and client coding along with the methods included in building this project. One of the things I did learn and really understood was the payloads and each payload having a specific part to the project.



Saved: 11/27/2025 4:41:14 PM

Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The easiest part of the assignment was setting up everything in the project. I was able to create the files needed and able to puse them in each class.



Saved: 11/27/2025 4:42:11 PM

⇒ Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part of this assignment was the processBattles method that I made for the onRoundEnd(). This method was really tough to do and I really struggled with it, but ended up making it so it would work in the code. I really had a hard time trying to fufill all the requirements needed with my code constantly breaking and I having to constantly fix it was a struggle. But in the end it taught me a lot and manage to complete it.



Saved: 11/27/2025 4:43:51 PM